

Développement Front

D'autres hooks

Hook - useId et forwardRef

- Générateur d'un id unique basé sur le ReactDOM
- C'est une sorte de useMemo avec un random.

```
const id = useId()
return <div>
  <label className="form-label" htmlFor={id}>{label}</label>
  <input
    ref={inputRef}
    id={id}
```

- Permet aussi de faire un point fixe pour éviter des recalculs
- Permet d'accéder à des propriétés d'objets html

Hook - useRef

- Référence à un élément html
- Peut être utilisé comme un useMemo
- À utiliser avec un useEffect

```
const ref = useRef(null)

useEffect(() => {
  console.log(ref.current.offsetHeight)
}, []);

return <div ref={ref}>
```


Hook personnalisé

- C'est une fonction qui utilise des hooks
 - Peut renvoyer autre chose qu'un tableau
 - On nomme la fonction use... par convention
- Permet de faire des fonctions de plus haut niveau
 - Ex : `const [checked, toggleChecked] = useToggle(false)`
 - Ex : `const {count, increment, decrement} = useIncrement({base: 0, max: 10, min: 0})`
 - Ex : `useFetch`, fonction de chargement de fichier
- Permet d'isoler une partie de la logique de l'application
- Il existe des hooks personnalisés en ligne
 - <https://usehooks.com> par exemple
 - react-use sur GitHub : Exemple d'implémentations en TypeScript

Les quatre saisons avec un hook personnalisé

- On déplace le comportement dans un hook useFourSeasonsBehaviour
- Implémentation de type MVC

Hook - useReducer

- Dédié aux états complexes avec mutations importantes
- Ex : ToDo list avec ajout, suppression, modification, archivage, ... des tâches
- On utilise une façade
- Le useReducer
 - {state, dispatch} où le dispatch transfère le traitement au reducer
- Mise en place d'une fonction reducer
 - Paramètres {state, action} où action est une constante venant de l'utilisation de dispatch
- ATTENTION : pour être sûr que React détecte les changements
 - Il ne faut pas muter les objets
 - Il faut toujours créer un nouvel objet à partir de l'ancien

Les quatre saisons avec useReducer

- On implémente useFourSeasonsBehaviour avec un useReducer à l'intérieur

Quelques fonctions de plus

Les portails

- Permet de créer des composants qui seront attachés à un élément du DOM autre que le composant parent
- Pratique pour les popups et autres boîtes modales
- Fonction `createPortal` de react-dom

```
function Modal () {  
  return createPortal(  
    <div style={{  
      position: 'absolute',  
      top: 0,  
      right: 0,  
      padding: 10,  
      border: 'solid 1px grey',  
      background: '#FFF'  
    }}>  
      Je suis une modale  
    </div>, document.body)  
  )  
}
```

ErrorBoundary- Gestion des erreurs

- Empêcher que la propagation d'erreur plante toute l'application
 - Ne laisser planter que le strict minimum
 - Prévoir un élément de substitution
- Utilisation d'un composant « decorator »
 - ErrorBoundary <https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>
 - Code ancienne mode, mais toujours d'actualité
 - Création du composant ErrorBoundary
 - Utilisation de ce composant comme décorateur

```
<ErrorBoundary fallback={<p>Impossible d'afficher la liste produit</p>}>  
  <ProductTable products={visibleProducts}/>  
</ErrorBoundary>
```

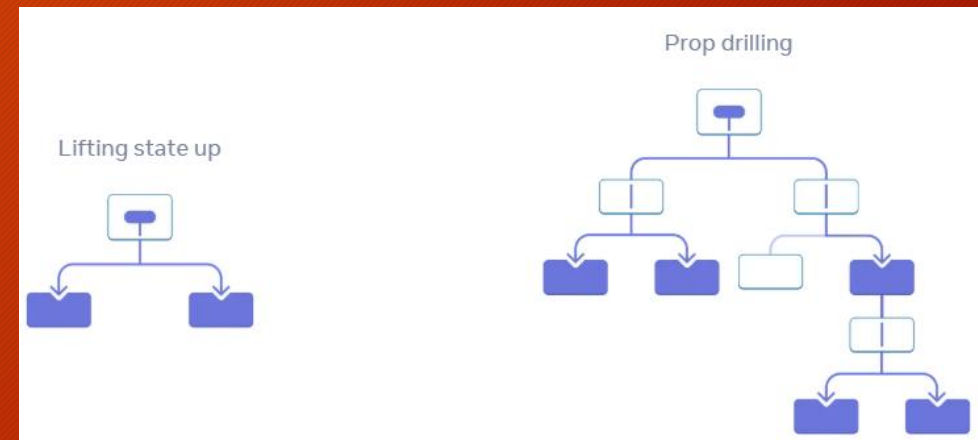

Fonction lazy et décorateur Suspense

- Contexte : chargement de composant React lourd
- Solution : chargement asynchrone

```
const SingleLazy = lazy(() => import('./pages/Single'))  
return <Suspense fallback={<div>Chargement des composants en cours</div>}>  
  <SingleLazy postId={param}/>  
</Suspense>;
```


Le hook useContext

- Eviter le props drilling
- Moyen d'exporter des composants dans toute l'appli
- createContext / useContext
- Et un décorateur sur les composants clients
- Utile pour
 - Langue du site
 - Thème
 - ...
- C'est un annuaire



React Routeur

- Permet de créer une structure virtuelle de navigation par url
- <https://reactrouter.com/tutorials/quickstart>

Aides de l'environnement

Navigateur

- Console : Passer en mode « Verbose » permet de voir les avertissements de React
 - Lenteur => useMemo, etc.
- Extension React Developer Tools du navigateur
 - Exploration des composants
 - Profiler pour les performances de la page

js doc

- `// @ts-check`
- Commentaires style javaDoc
- Permet de « typer » des fonctions (contrôle de eslint)
- Documentation accessible pendant l'utilisation de l'environnement

Best Practices

Best practices #1

- https://www.youtube.com/watch?v=6s_T8mlt0_Y
- Préférer Fragment (ou <> (autre écriture)) à div
 - Donne plus de souplesse pour l'intégration des composants
 - Facilite l'accessibilité en enlevant des nœuds inutiles
 - DOM plus petit donc gain performances
 - ATTENTION : les fragments ne supportent pas les styles

Best practices #2

- Limiter le code js dans jsx
 - Préférer l'utilisation de fonction pour les traitements complexes
 - Améliore la lisibilité, maintenabilité et la modularité
 - Séparation des concepts

```
const Counter = () => {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button  
      onClick={() => {  
        setCount(count + 1);  
        alert('You clicked the button!');  
      }}  
    >  
      Click me  
    </button>  
  );  
};
```



```
const Counter = () => {  
  const [count, setCount] = useState(0);  
  const handleClick = () => {  
    setCount(count + 1);  
    alert('You clicked the button!');  
  }  
  
  return (  
    <button  
      onClick={handleClick}  
    >  
      Click me  
    </button>  
  );  
};
```



Best practices #3

- Eviter les useEffect
 - Il existe souvent une façon d'implémenter sans hook.

Best practices #4

- Faire attention à ce qui est recalculé à chaque re-render
 - Penser à sortir les fonctions de dispatch par exemple

Best practices en vrac (1/2)

- Use Memoization for Expensive Calculations:
 - React provides the `React.memo()` higher-order component and `useMemo()` hook for memoizing the results of expensive calculations. This can improve performance by preventing unnecessary re-renders of components.
- Avoid Arrow Functions in JSX Props:
 - Avoid using arrow functions directly in JSX props, as this can create a new function instance on each render. Instead, define the function outside of the render method and pass it as a prop.
- Use the React DevTools Extension:
 - Install the React DevTools browser extension for Chrome or Firefox. It provides a set of debugging tools specifically designed for React applications, allowing you to inspect component hierarchies, view props and state, and analyze performance.
- Use Conditional Rendering with Null or Fragment:
 - Instead of using ternary operators for conditional rendering, you can use null or React Fragments to conditionally render components. This can result in cleaner and more readable code.
- Optimize Component Re-renders with PureComponent:
 - Use React's `PureComponent` class for components that only re-render when their props or state change. `PureComponent` performs a shallow comparison of props and state to determine if a re-render is necessary, potentially improving performance.

Best practices en vrac (2/2)

- **Avoid Using Index as Key in Lists:**
 - Avoid using the array index as the key prop when rendering lists of components. Instead, use a unique identifier from your data, such as an ID or slug. Using the index as a key can lead to unexpected behavior when reordering or modifying the list.
- **Use React.forwardRef for Higher-Order Components:**
 - When creating higher-order components that need to pass refs to their wrapped components, use the React.forwardRef() function. This allows the higher-order component to forward refs to the underlying DOM elements.
- **Leverage Context API for Global State Management:**
 - Instead of using prop drilling to pass data down through multiple layers of components, consider using React's Context API for global state management. Context allows you to share data across components without explicitly passing props.
- **Use React.memo() for Functional Components:**
 - Similar to PureComponent for class components, React.memo() can be used to memoize functional components and prevent unnecessary re-renders. Wrap your functional components with React.memo() to optimize performance.

Ressources potentiellement intéressantes

Il y a de tout !

Zustand

- <https://www.youtube.com/watch?v=JRGMe2Zq0k>
- Package js pour créer un store global

Heroic Icons

- <https://heroicons.com/solid>